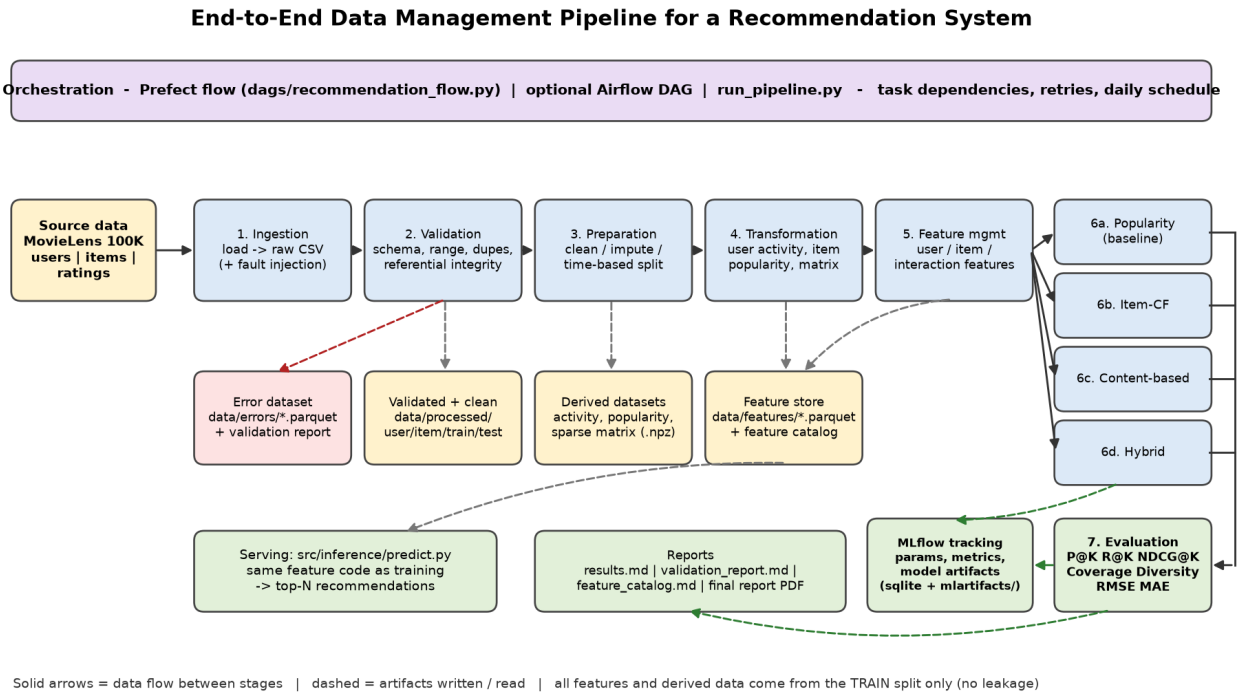
End-to-End Data Management Pipeline for a Recommendation System

Data Management for Machine Learning (AIMLZG529) - Assignment 2 | Rajavarman Mani | Roll No. 2025ag05218

1. Objective and dataset

The goal is a clean, modular and reproducible pipeline that takes raw user, item and interaction data to evaluated recommenders: ingestion, validation, preparation, transformation, feature management, training, evaluation and orchestration. I used **MovieLens 100K** (943 users, 1,682 movies, 99,540 ratings after cleaning; 93.72% sparse). The task is top-10 movie recommendation; a rating of 4 or more counts as a positive signal. The full source code, configuration and instructions are in the GitHub repository (see README).

2. Architecture and tools



Tools: Python/pandas/NumPy/SciPy for processing; Parquet for storage (typed, compressed, fast); a custom, file-based feature layer; MLflow (SQLite backend) for experiment tracking; Prefect for orchestration (an equivalent Airflow DAG is included); pytest for unit tests. Everything is driven by one `configs/config.yaml` and a fixed seed, so every run is reproducible.

3. Design decisions per stage

- **Ingestion.** Data is loaded as untyped text to mimic an external source. MovieLens is almost perfectly clean, so a *seeded, documented* fault-injection step corrupts about 657 raw rows (duplicates, invalid ratings/timestamps, missing values, orphan ids). This gives the validator real defects and allows an injected-vs-detected check; it can be disabled in the config.
- **Validation.** Checks cover missing values, data types, rating range (1-5), timestamp window, duplicates and referential integrity. Defects are split by severity: unusable records are **rejected** to an error dataset with the reason, while repairable attribute problems (e.g. age 250, missing title) are logged as warnings and fixed in preparation, so we do not throw away good interactions because of a bad profile field. A run fails if more than 20% of a table is rejected.
- **Preparation and split.** Imputation (median), text/case standardisation, UTC timestamps and one rating per (user, item). A **time-based split** places the most recent 20% of each user's history in the test set (79,996 train / 19,544 test rows). Per-user chronological splitting keeps nearly every user evaluable, which a single global cut-off would not on a 7-month dataset (a global option is available).
- **Transformation and features.** User activity summary, item popularity and a sparse 943 x 1682 rating matrix are built from **train only**. Features are documented in a registry that generates the catalog, are computed point-in-time at the last training timestamp, and use one set of functions for both training and prediction (`src/inference/predict.py`), which avoids training/serving skew. I chose a custom feature layer over Feast because the requirement is reuse, which this provides without extra infrastructure.

- **Models.** (a) *Popularity* baseline: the most-liked items for everyone. (b) *Item-CF*: item-item cosine similarity on the liked matrix with significance shrinkage co/(co+10), plus an adjusted-cosine k-NN rating predictor. (c) *Content-based*: cosine between the user's genre-preference features and the item's genres. (d) *Hybrid*: 0.7 x CF + 0.3 x content scores, each scaled per user.
- **Orchestration.** A Prefect flow runs ingest, validate, prepare, transform and features sequentially, then the four training tasks in parallel, then evaluation. Dependencies are explicit, ingestion retries once, and a cron schedule can be attached with `--serve`.

4. Data validation outcome

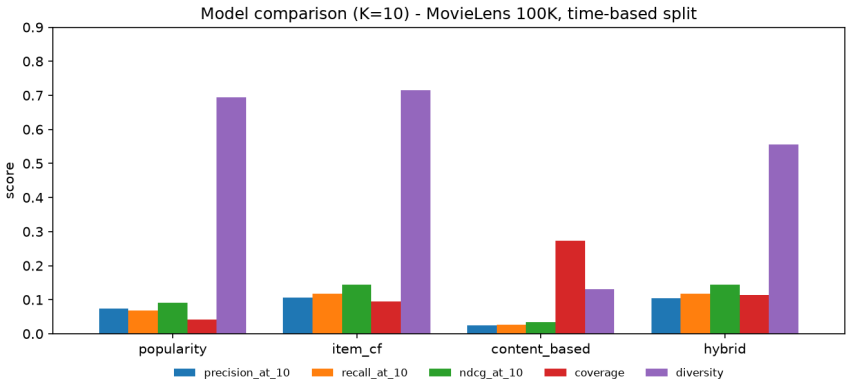
Table	Rows in	Valid	Rejected	Main reject reasons
interactions	100,150	99,540	610 (0.61%)	duplicate_row (150), invalid_rating (100), orphan_user_id (70), orphan_item_id (70), invalid_timestamp (64), missing_value:rating (60), wrong_dtype:rating (50), missing_value:user_id (30), wrong_dtype:timestamp (16)
users	953	943	10 (1.05%)	duplicate_key:user_id (10)
items	1,687	1,682	5 (0.30%)	duplicate_key:item_id (5)

All injected faults were detected (the report `reports/validation_report.md` lists injected vs detected per fault type); rejected rows are stored in `data/errors/`. The feature catalog (user, item and interaction features) is in `reports/feature_catalog.md`.

5. Results

Evaluation uses K = 10 on 904 users who have at least one liked item in their held-out period; candidates are all items the user has not rated in training. Coverage is the share of the catalogue that appears in any list; diversity is the mean pairwise genre dissimilarity inside a list.

Model	P@10	R@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity (baseline)	0.0731	0.0691	0.0919	0.0422	0.6946	1.0808	0.8683
Item-CF	0.1054	0.1174	0.1448	0.0957	0.7155	1.0571	0.8379
Content-based	0.0248	0.0259	0.0345	0.2735	0.1302	n/a	n/a
Hybrid (CF + content)	0.1051	0.1181	0.1440	0.1136	0.5565	1.0571	0.8379



Findings. The personalised Item-CF model clearly beats the popularity baseline: NDCG@10 0.145 vs 0.092 (+58%), Precision@10 0.105 vs 0.073 (+44%), and it also recommends a wider part of the catalogue (9.6% vs 4.2%). Its rating predictions are better too (RMSE 1.057 vs 1.081), although that gap is small, because item averages are already a strong rating baseline. The hybrid matches Item-CF on accuracy (NDCG 0.144) but trades diversity (0.56 vs 0.72) for higher coverage (11.4%): blending in genre similarity pulls lists toward the user's favourite genres. The content-only model is weak on accuracy (P@10 0.025) because 19 coarse genres cannot separate items well, yet it has the highest coverage (27.3%) as it is not dominated by popular titles. With a single dataset and seed these differences are indicative, not statistically tested.

6. Limitations and future work

- **Cold start:** users and items without training history get no personalised scores; item-CF cannot recommend the 71 items that never appear in training. Richer item metadata or an embedding model would help.
- **Evaluation:** offline top-K metrics on one time-based split and one seed; no confidence intervals or A/B test. The per-user split lets one user's test events precede other users' training events (mild cross-user temporal overlap); the global split removes this at the cost of evaluable users.
- **Scale:** dense user x item score matrices are fine for 943 x 1,682 but not for millions of items; production would need sparse retrieval or approximate nearest neighbours, and an online feature store with freshness guarantees.

- **Data realism:** faults are injected, so validation scores demonstrate the checks rather than real-world data quality; thresholds (e.g. rating >= 4 as positive) are assumptions. Tuning (neighbours, shrinkage, blend weights) was not performed.
- **Operations:** orchestration is local (Prefect ephemeral/server); drift monitoring, model registry promotion and incremental ingestion are natural next steps.